

DEVTRACK AI

Cloud-Native Job Application and Career Management Platform Using DevOps Practices and AWS

SYNOPSIS REPORT

Submitted By: Rohit

Registration No.: 12322037

Roll No.: 60

Section: K302344

Submitted To: Mr. Ajay Kumar Badhan



Department of Computer Science and Engineering
Lovely Professional University
Punjab, India

CONTENTS

1.PROJECT DESCRIPTION

Overview of DevTrack AI, its purpose, problem domain, and cloud-native approach.

2. OBJECTIVES AND SCOPE

Key academic objectives and the functional and technical scope of the proposed project.

3. PROPOSED SYSTEM AND TECHNOLOGY STACK

Core application modules and the technologies used for development, DevOps, cloud deployment, and monitoring.

4. COMPLETE PROJECT PERSPECTIVE AND SYSTEM ARCHITECTURE

End-to-end view of local development, source control, automation, AWS deployment, Kubernetes, and monitoring.

5. WORKING FLOW AND DEVOPS ARCHITECTURE

Application request flow, CI/CD pipeline, Kubernetes orchestration, load balancing, scaling, and self-healing.

6. DATA FLOW DIAGRAMS

Context-level, application-level, DevOps operational, and monitoring/scaling data-flow representations.

7. IMPLEMENTATION AND DEPLOYMENT PLAN

Step-by-step plan covering development, containerization, infrastructure provisioning, AWS deployment, CI/CD, and monitoring.

8. EXPECTED OUTCOMES AND CONCLUSION

Expected technical outcomes, project value, and overall conclusion.

.

1. PROJECT DESCRIPTION

DevTrack AI is a cloud-native Job Application and Career Management Platform designed to help users organize and track their job-search activities through a centralized web application. The system will manage companies, job applications, application status, interview information, notes, and career progress through an interactive dashboard.

The project is designed not only as a full-stack application but also as an end-to-end DevOps implementation. The application will be developed using React.js, FastAPI, and PostgreSQL, packaged with Docker, orchestrated with Kubernetes, automated through GitHub Actions, provisioned using Terraform, monitored with Prometheus and Grafana, and deployed in an AWS cloud environment.

For the cloud demonstration, a lightweight Kubernetes environment using K3s on AWS EC2 is proposed. This allows the project to demonstrate real Kubernetes concepts on AWS, including deployments, services, load balancing, scaling, and self-healing, while keeping the architecture suitable for an academic project.

2. OBJECTIVES AND SCOPE

The main objective is to demonstrate the complete lifecycle of a modern cloud-native application, from development and containerization to automated deployment and monitoring.

- Develop a functional web platform for managing job applications and career activities.
- Containerize the application using Docker.
- Deploy the application on AWS and demonstrate cloud-based access.

- Implement Kubernetes orchestration using K3s, including Pods, Deployments, Services, scaling, load balancing, and self-healing.
- Use Terraform to define and provision cloud infrastructure.
- Automate build and deployment activities using GitHub Actions.
- Monitor application and infrastructure metrics using Prometheus and Grafana.

3. PROPOSED SYSTEM AND TECHNOLOGY STACK

Core Application Modules

- Job Application Management – create, update, delete, and organize applications.
- Company and Role Tracking – maintain company, role, location, source, and notes.
- Status and Interview Tracking – manage stages, interview rounds, schedules, and outcomes.
- Dashboard and Analytics – present application progress and useful summaries.

Technology Stack

Layer	Technology
Frontend	React.js
Backend	FastAPI
Database	PostgreSQL
Containerization	Docker
Orchestration	Kubernetes / K3s
Cloud Platform	Amazon Web Services (AWS)
Infrastructure as Code	Terraform
CI/CD	GitHub Actions
Monitoring	Prometheus
Visualization	Grafana

4. COMPLETE PROJECT PERSPECTIVE AND SYSTEM ARCHITECTURE

The following diagram presents the complete project from a practical perspective. Development and validation take place locally, source code is managed through GitHub, automation is handled through GitHub Actions, and the final cloud environment runs on AWS. Kubernetes manages the deployed application, while Prometheus and Grafana provide operational visibility.

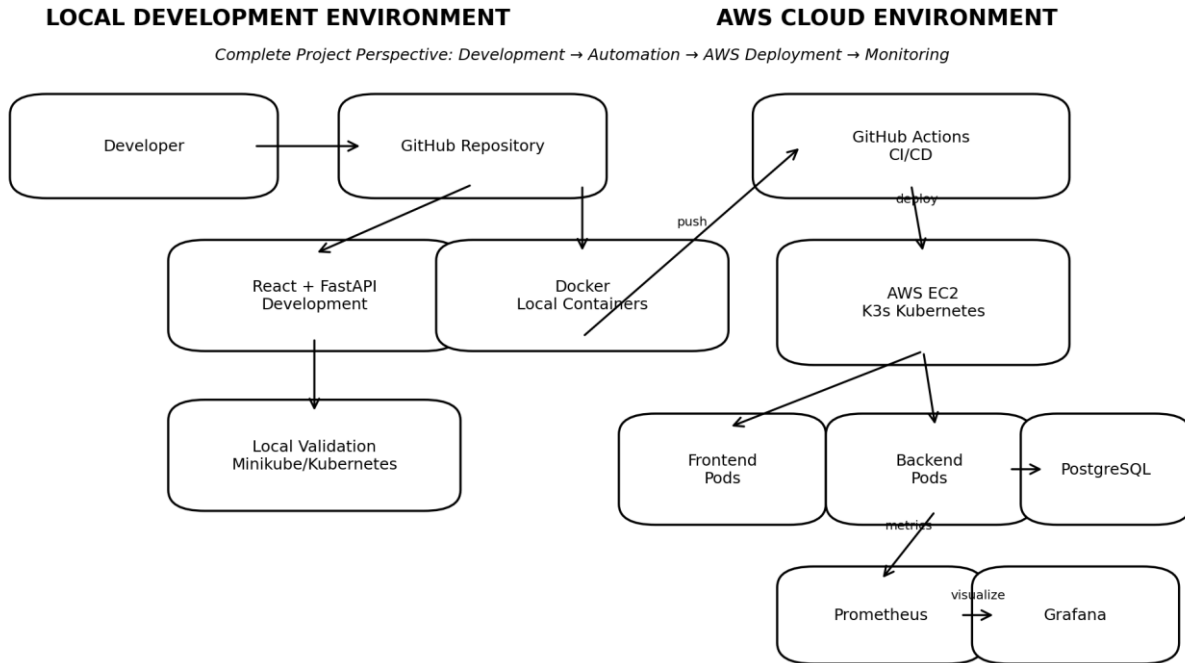


Figure 1: Complete DevTrack AI Perspective – Local Development and AWS Deployment

Architecture Components

- Local Environment: application development, Docker packaging, and initial Kubernetes validation.
- GitHub: centralized version control and source-code management.
- GitHub Actions: automated build, test, and deployment workflow.
- AWS EC2 + K3s: cloud-hosted Kubernetes environment for the final application demonstration.
- Prometheus and Grafana: collection and visualization of operational metrics.

5. WORKING FLOW AND DEVOPS ARCHITECTURE

5.1 Application Working Flow

APPLICATION REQUEST HANDLING WORKFLOW

User actions are processed through APIs, persisted in PostgreSQL, and returned as dashboard updates.

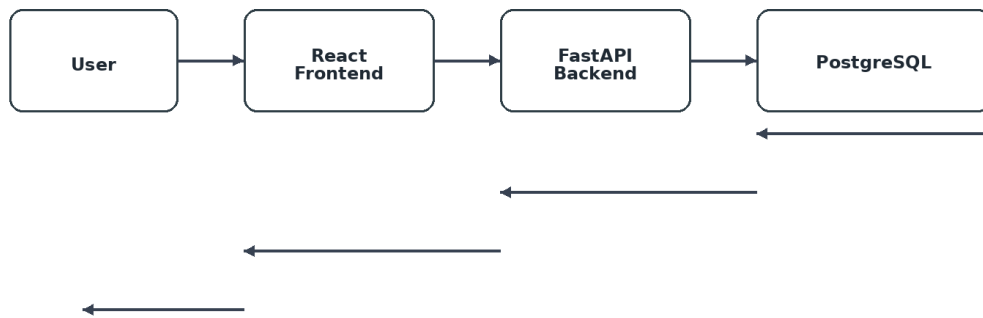


Figure 2: Application Request and Response Flow

Users interact with the React frontend, which communicates with the FastAPI backend through API requests. The backend processes the request and reads or updates application information stored in PostgreSQL. The response is then returned to the frontend.

5.2 CI/CD Workflow

CI/CD WORKFLOW — GITHUB ACTIONS



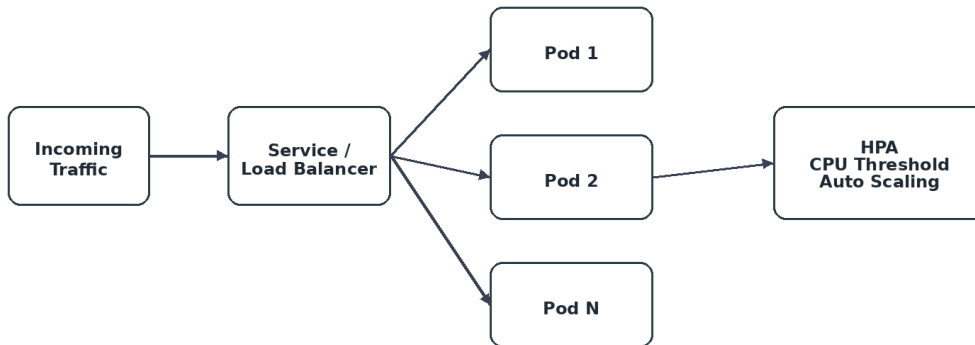
Automated flow: Code → Test → Build → Image → Deploy → Verify

Figure 3: Continuous Integration and Deployment Workflow

Code changes pushed to GitHub trigger the configured automation pipeline. The workflow can validate the application, execute tests, build Docker images, and deploy updated workloads to the Kubernetes environment.

5.3 Kubernetes Reliability and Scaling

KUBERNETES SCALING, LOAD BALANCING AND SELF-HEALING



Deployment controller recreates failed pods automatically; HPA increases or decreases replicas based on configured resource

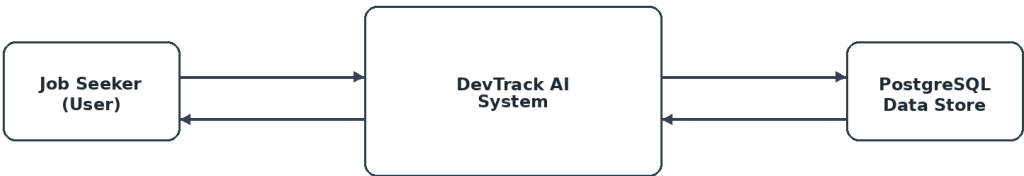
Figure 4: Kubernetes Load Balancing, Scaling and Self-Healing

Kubernetes Services distribute requests across healthy pods. Deployments maintain the desired number of replicas, Horizontal Pod Autoscaling can adjust replicas according to configured resource thresholds, and failed pods are replaced to restore the desired state.

6. DATA FLOW DIAGRAMS

6.1 DFD Level 0 – Context Diagram

DFD LEVEL 0 – CONTEXT DIAGRAM



Application details, interview information and analytics requests flow through the system and are stored securely.

Figure 5: DFD Level 0 – DevTrack AI Context Diagram

The user exchanges job application and career-management information with DevTrack AI. The system processes these requests and stores persistent information in the PostgreSQL database.

6.2 DFD Level 1 – Application Process Flow

DFD LEVEL 1 – DEVTRACK AI PROCESS FLOW

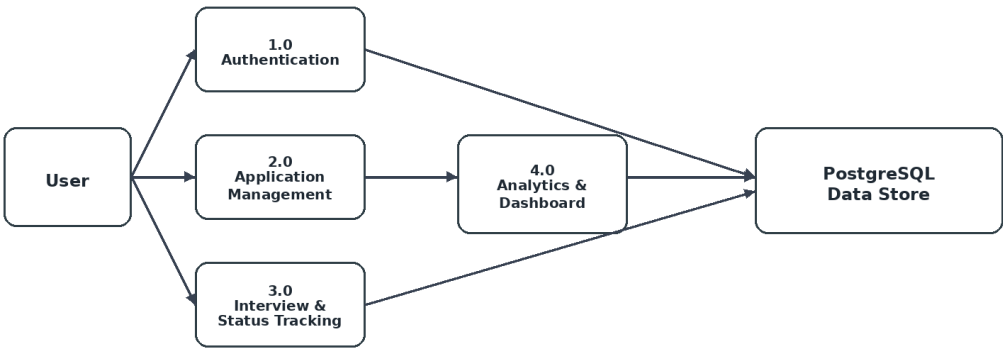


Figure 6: DFD Level 1 – Detailed Application Data Flow

The application is decomposed into major processes such as application management, status and interview tracking, and dashboard generation. These processes interact with the data store to process user requests.

6.3 DFD – DevOps Tools and Operational Flow

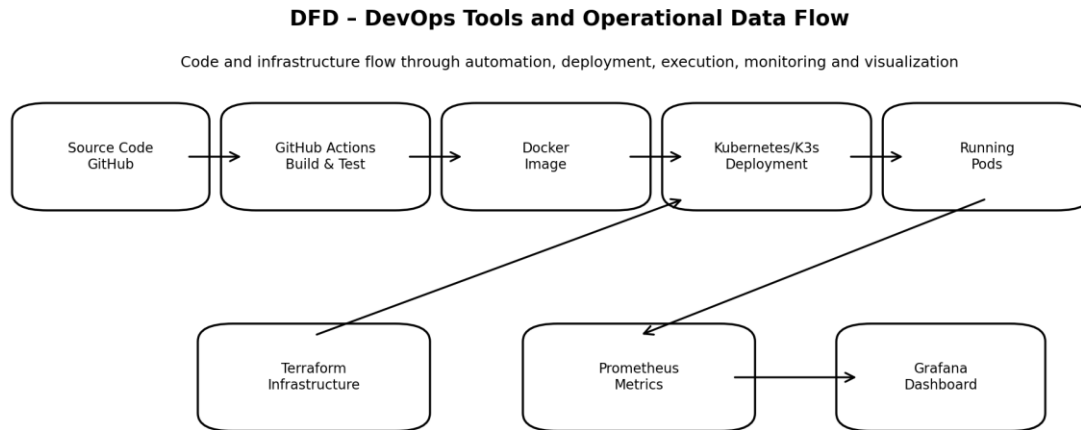


Figure 7: DevOps Tools and Operational Data Flow

This diagram explains how the DevOps tools work together. Source code flows from GitHub through automated CI/CD stages, Docker packages the application, Kubernetes deploys and manages the workloads, Terraform supports infrastructure provisioning, and Prometheus and Grafana provide operational monitoring.

6.4 DFD – Scaling and Monitoring Flow

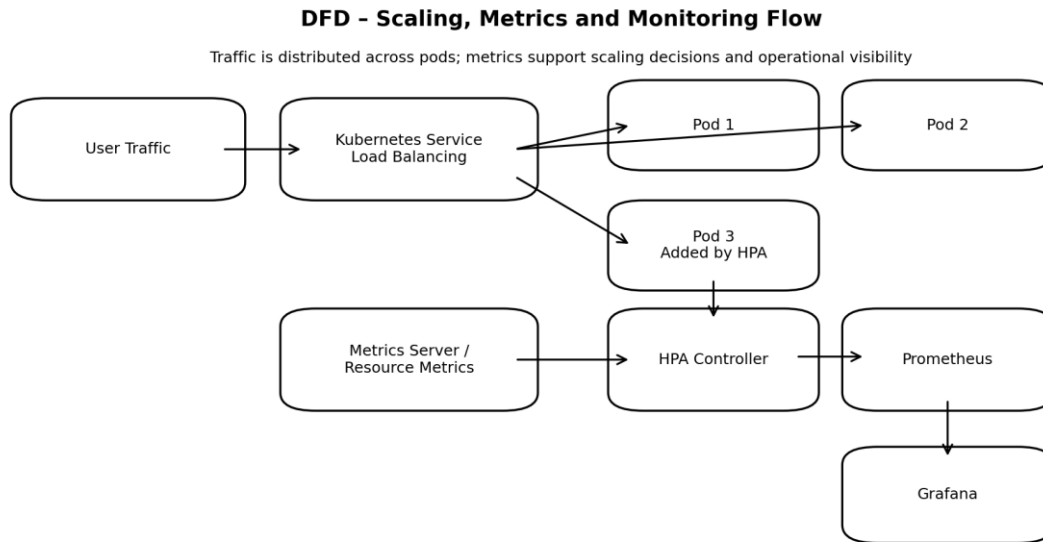


Figure 8: Scaling, Metrics and Monitoring Data Flow

Incoming traffic reaches the Kubernetes Service and is distributed across application pods. Resource metrics support scaling decisions through the Horizontal Pod Autoscaler, while Prometheus collects operational data and Grafana presents the results through dashboards.

7. IMPLEMENTATION AND DEPLOYMENT PLAN

Application Development: Build the React frontend, FastAPI backend, and PostgreSQL data model.

Containerization: Create Docker images and verify consistent execution of the application services.

Kubernetes Configuration: Create manifests for Deployments, Services, replicas, configuration, and required resources.

Reliability Demonstration: Demonstrate multiple pods, service-based load balancing, Horizontal Pod Autoscaling, and self-healing.

Infrastructure as Code: Use Terraform to define the AWS infrastructure required for deployment.

AWS Deployment: Deploy the application to AWS EC2 and configure the K3s Kubernetes environment.

CI/CD Integration: Connect GitHub Actions with the repository and deployment workflow.

Monitoring: Configure Prometheus metrics collection and Grafana dashboards for the final demonstration.

8. EXPECTED OUTCOMES

- A fully functional Job Application and Career Management Platform.
- A working AWS cloud deployment accessible for academic demonstration.
- Docker-based application containerization.
- A Kubernetes/K3s environment running real application workloads on AWS.
- Demonstration of multiple pods, service-based load balancing, auto-scaling, and self-healing.
- Terraform-based Infrastructure as Code.
- An automated CI/CD workflow using GitHub Actions.
- Prometheus-based monitoring and Grafana dashboards.
- A complete portfolio project demonstrating practical full-stack, cloud, and DevOps skills.

CONCLUSION

DevTrack AI combines full-stack development with practical cloud and DevOps engineering in one integrated project. The proposed system demonstrates how an application can move from source code to containerized deployment, automated delivery, Kubernetes orchestration, cloud hosting, and continuous monitoring.

The project provides hands-on exposure to widely used industry technologies including Docker, Kubernetes, Terraform, GitHub Actions, Prometheus, Grafana, and AWS. By implementing the complete workflow and demonstrating the architecture in a real cloud environment, DevTrack AI is expected to provide strong academic value and a substantial project for placement and technical interviews.